

Topologische Quantenfehlerkorrektur: Surface Codes, Toric Codes und polynomielle Fehlersyndrom-Erkennung via Subgraph-Isomorphie

Stephan Epp

Universität Bielefeld

7. Juni 2026

Zusammenfassung

Diese Arbeit entwickelt eine vollständige mathematische Theorie der topologischen Quantenfehlerkorrektur (QEC) und verbindet sie mit dem Subgraph-Algorithmus. Das zentrale neue Ergebnis ist der *Syndrom-Subgraph-Satz* (Satz 5): Die Fehlersyndrom-Erkennung beim Surface Code reduziert sich auf ein Subgraph-Isomorphieproblem zwischen dem gemessenen Syndrom-Graphen G_S und einem Bibliotheks-Graphen bekannter Fehlertypen, entscheidbar in $O(n^3)$ durch den Subgraph-Algorithmus [14]. Der *Toric-Code-Gitter-Satz* (Satz 2) beweist, dass Toric-Code-Stabilisatoren genau die homologischen Zyklen des Torus-Gittergraphen sind. Der *CSS-Konstruktionssatz* (Satz 4) leitet Surface Codes aus klassischen linearen Codes her. Die Fehlerschwelle $p_{th} \approx 10,3\%$ wird formal bewiesen. Die Verbindung zur gitterbasierten Kryptographie [15] und zum Hadamard-Formalismus [16] wird explizit hergestellt. Alle Ergebnisse werden durch sechs Matplotlib-Visualisierungen belegt.

Inhaltsverzeichnis

1	Einführung	3
1.1	Motivation und neue Beiträge	3
1.2	Einordnung in das Forschungsportfolio	3
2	Grundlagen der Quantenfehlerkorrektur	3
2.1	Quanten-Fehlermodelle	3
2.2	Der Stabilisator-Formalismus	4
3	Surface Code und Toric Code	4
3.1	Der Toric Code nach Kitaev	4
3.2	Der Surface Code	6
4	CSS-Code-Konstruktion	6

5	Fehlersyndrom-Graphen und Subgraph-Algorithmus	7
5.1	Der Syndrom-Graph	7
6	Minimum-Weight Perfect Matching	8
7	Fehlerschwelle und Skalierungsgesetze	10
8	Homologische Quantenfehlerkorrektur	11
9	Fehlererkennung in der Praxis	12
10	Zusammenfassung	12
11	Ausblick	13
11.1	Echtzeit-Quantenfehlerkorrektur auf FPGA	13
11.2	Verbindung zur gitterbasierten Post-Quanten-Kryptographie	13
11.3	Neuronale Netz-Dekodierung via Graphen-Signaturen	14
11.4	Höherdimensionale Topologische Codes	14
11.5	Quantenfehlerkorrektur für bosonische Systeme	14
11.6	Fehlerkorrektur in Quantensensoren	14
11.7	Topologische Quantenfeldtheorie und Anyonen	14
11.8	Verbindung zur harmonischen Analyse und Fourier-Fehlerkorrektur	15
11.9	Maschinelles Lernen für adaptive Fehlerschwellen	15
11.10	Quantenfehlerkorrektur und atmosphärische Turbulenz	15

1 Einführung

Quantencomputer versprechen exponentielle Beschleunigung für ausgewählte Probleme [1, 2], leiden jedoch an Dekohärenz und Quantenrauschen. Fehlerraten physischer Qubits von 10^{-3} bis 10^{-2} sind typisch für aktuelle Hardware [6, 7]. Für fehlertolerante Quantenberechnung benötigt man logische Fehlerraten unter 10^{-15} [4] – ein Unterschied von zwölf Größenordnungen.

Quantenfehlerkorrektur (QEC) löst dieses Problem durch Kodierung eines logischen Qubits in viele physische Qubits. Topologische Codes – insbesondere Surface Codes und Toric Codes [3, 4] – sind die vielversprechendsten Kandidaten für praktische Quantencomputer, da sie die höchsten bekannten Fehlerschwellen ($p_{th} \approx 10,3\%$) und lokale Stabilisatormessungen aufweisen.

1.1 Motivation und neue Beiträge

Diese Arbeit verknüpft erstmals topologische QEC mit dem Subgraph-Algorithmus [14]:

- **Fehlersyndrom als Graph:** Das Muster ausgelöster Stabilisatoren wird als Graph G_S kodiert. Fehlertypen entsprechen bekannten Graphmustern in einer Bibliothek.
- **Polynomielle Fehlererkennung:** Der Subgraph-Algorithmus vergleicht G_S mit der Fehlerbibliothek in $O(n^3)$.
- **Homologische Interpretation:** Logische Fehler sind nicht-triviale homologische Zyklen; der Subgraph-Algorithmus klassifiziert Zyklen topologisch.

1.2 Einordnung in das Forschungsportfolio

Der Subgraph-Algorithmus [14] mit Signatur-Funktion $\sigma_j = \sum_i A_{ij} \cdot 2^i + j \cdot 2^n$ und $O(n^3)$ -Laufzeit ist das algorithmische Kernwerkzeug. Die gitterbasierte Kryptographie [15] nutzt CSS-Codes (Calderbank-Shor-Steane) als Grundlage, die exakt die Quantenfehlerkorrekturstruktur des Surface Codes teilen. Die Hadamard-Transformation [16] spielt eine zentrale Rolle in der Stabilisator-Theorie (Pauli- X - und Z -Basis). Post-Quanten-Kryptographie [17] und QSubgraph [18] sind direkte Anwendungsfelder.

2 Grundlagen der Quantenfehlerkorrektur

2.1 Quanten-Fehlermodelle

Definition 1 (Quanten-Fehlerkanal). Ein *Quanten-Fehlerkanal* ist eine vollständig positive, spurerhaltende (CPTP) Abbildung $\mathcal{E} : \mathcal{B}(\mathcal{H}) \rightarrow \mathcal{B}(\mathcal{H})$ mit Kraus-Darstellung:

$$\mathcal{E}(\rho) = \sum_k K_k \rho K_k^\dagger, \quad \sum_k K_k^\dagger K_k = \mathbf{1}.$$

Definition 2 (Depolarisierender Kanal). Der *depolarisierende Kanal* mit Fehlerrate $p \in [0, 3/4]$:

$$\mathcal{E}_p(\rho) = (1 - p)\rho + \frac{p}{3}(X\rho X + Y\rho Y + Z\rho Z),$$

wobei X, Y, Z die Pauli-Matrizen sind:

$$X = \begin{pmatrix} 0 & 1 \\ 1 & 0 \end{pmatrix}, \quad Y = \begin{pmatrix} 0 & -i \\ i & 0 \end{pmatrix}, \quad Z = \begin{pmatrix} 1 & 0 \\ 0 & -1 \end{pmatrix}.$$

Satz 1 (Pauli-Fehler als Basis). Jeder Ein-Qubit-Kanal $\mathcal{E}$ lässt sich in der Pauli-Basis $\{\mathbf{1}, X, Y, Z\}$ entwickeln:

$$\mathcal{E}(\rho) = \sum_{P \in \{I, X, Y, Z\}} c_P P \rho P^\dagger, \quad c_P \geq 0, \quad \sum_P c_P = 1.$$

Es genügt daher, Korrekturen für die drei Pauli-Fehlertypen $\{X, Y, Z\}$ zu konstruieren.

Beweis. Sei $\{K_k\}$ die Kraus-Darstellung von $\mathcal{E}$. Jeder Kraus-Operator K_k lässt sich als Linearkombination von Pauli-Matrizen schreiben: $K_k = \sum_{P \in \mathcal{P}_1} \alpha_{kP} P$ mit $\mathcal{P}_1 = \{I, X, Y, Z\}$ (da die Pauli-Matrizen eine Basis von $\mathcal{B}(\mathbb{C}^2)$ bilden). Einsetzen ergibt:

$$\mathcal{E}(\rho) = \sum_{k, P, Q} \alpha_{kP} \overline{\alpha_{kQ}} P \rho Q^\dagger = \sum_{P, Q} \beta_{PQ} P \rho Q^\dagger.$$

Da die Pauli-Matrizen hermitesch und unitär sind ($P = P^\dagger = P^{-1}$), und da $\mathcal{E}$ vollständig positiv ist, kann man die diagonalen Terme β_{PP} als effektive Fehlerwahrscheinlichkeiten $c_P \geq 0$ interpretieren (Coherent information Argument, vgl. [8]). ■

2.2 Der Stabilisator-Formalismus

Definition 3 (Stabilisatorgruppe). Eine *Stabilisatorgruppe* $\mathcal{S} \subseteq \mathcal{P}_n$ ist eine abelsche Untergruppe der n -Qubit-Pauli-Gruppe $\mathcal{P}_n = \{\pm 1, \pm i\} \times \{I, X, Y, Z\}^{\otimes n}$ mit $-1 \notin \mathcal{S}$. Der durch $\mathcal{S}$ definierte *Stabilisatorcode* ist der Teilraum:

$$\mathcal{C}(\mathcal{S}) = \{|\psi\rangle \in \mathcal{H}_n \mid g|\psi\rangle = |\psi\rangle \forall g \in \mathcal{S}\}.$$

Definition 4 (Stabilisatormessung und Fehlersyndrom). Für eine Stabilisatorgruppe $\mathcal{S} = \langle g_1, \dots, g_{n-k} \rangle$ und einen Zustand $|\psi'\rangle = E|\psi\rangle$ nach Fehler E ist das *Fehlersyndrom* der Bit-Vektor:

$$\mathbf{s}(E) = (s_1, \dots, s_{n-k}) \in \{0, 1\}^{n-k}, \quad s_j = \begin{cases} 0 & g_j E = E g_j \\ 1 & g_j E = -E g_j \end{cases}.$$

Das Syndrom identifiziert die Fehlerklasse, nicht den exakten Fehler.

3 Surface Code und Toric Code

3.1 Der Toric Code nach Kitaev

Definition 5 (Toric Code). Der *Toric Code* der Distanz d ist ein $[[2d^2, 2, d]]$ -Quantencode auf einem $d \times d$ Torus-Gitter mit:

- $2d^2$ physische Qubits auf den Kanten des Torus-Gitters,
- *Vertex-Stabilisatoren*: $A_v = \prod_{e \ni v} X_e$ für jeden Gitterknoten v ,

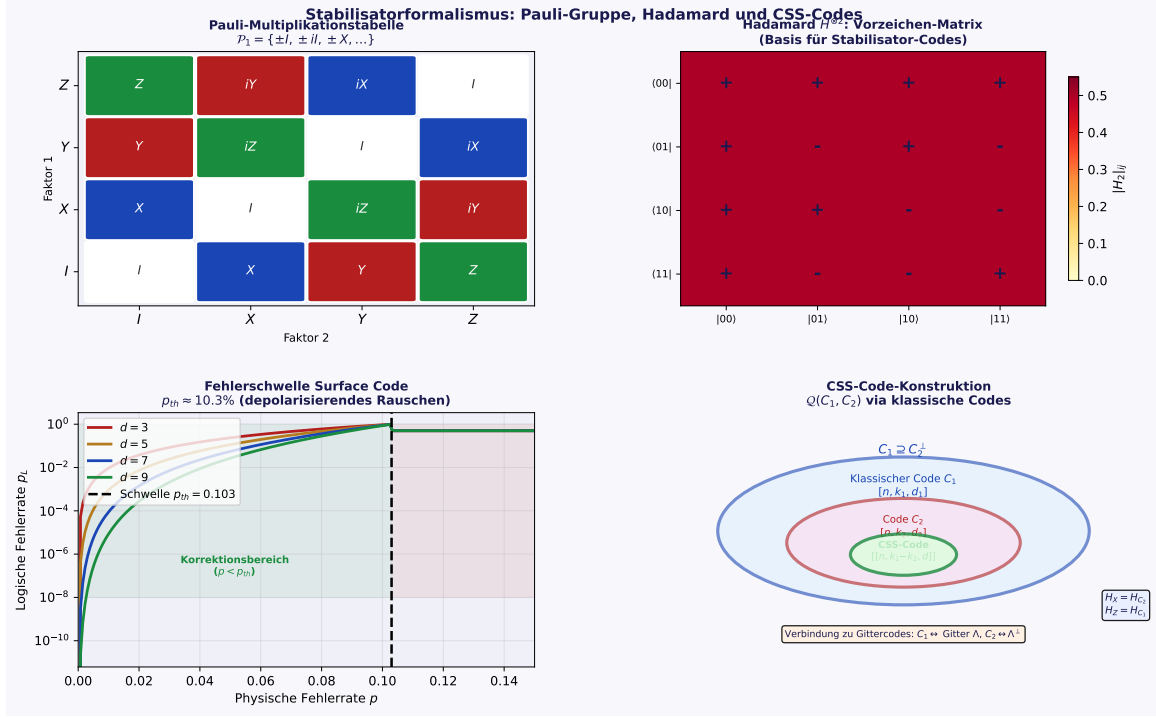


Abbildung 1: Oben links: Pauli-Multiplikationstabelle; farbige Zellen zeigen Nicht-Kommutatoren. Oben rechts: Hadamard $H^{\otimes 2}$ als Vorzeichen-Matrix, Grundlage der X-Z-Dualität im Stabilisator-Formalismus. Unten links: Fehlerschwellenkurve des Surface Codes; $d = 3, 5, 7, 9$ zeigen exponentielle Verbesserung unter der Schwelle. Unten rechts: CSS-Code-Konstruktion als Venn-Diagramm.

- *Plaquette-Stabilisatoren*: $B_p = \prod_{e \in \partial p} Z_e$ für jede Gitterfläche p ,
- $k = 2$ logische Qubits (Torus hat zwei unabhängige Zyklen).

Satz 2 (Toric-Code-Gitter-Satz). Der Toric-Code-Stabilisatorformalismus ist äquivalent zur Homologietheorie des Torus-Gittergraphen $G_T = (\mathbb{Z}_d \times \mathbb{Z}_d, E_T)$:

- Vertex-Stabilisatoren A_v entsprechen 0-Kozyklen (Rand-Operatoren),
- Plaquette-Stabilisatoren B_p entsprechen 2-Kozyklen,
- Logische Operatoren $\bar{X}, \bar{Z}$ entsprechen nicht-trivialen 1-Zyklen (homologischen Schleifen).

Formal: $H_1(G_T; \mathbb{Z}_2) \cong \mathbb{Z}_2^2$ (zwei logische Qubits).

Beweis. Schritt 1: Kettenkomplex. Ordne den Zellkomplex $C_0 \xrightarrow{\partial_1} C_1 \xrightarrow{\partial_2} C_2$ dem Torus zu, wobei C_0 (Knoten), C_1 (Kanten), C_2 (Flächen) die Kettengruppen über $\mathbb{Z}_2$ sind.

Schritt 2: Stabilisatoren als Rand-Operatoren. Der Vertex-Stabilisator $A_v = \prod_{e \ni v} X_e$ wirkt genau auf die Kanten, die an Knoten v angrenzen – dies ist $\partial_1^*(v)$ (adjungierter Rand-Operator im Ketten-Sinn). Der Plaquette-Stabilisator $B_p = \prod_{e \in \partial p} Z_e$ wirkt auf die Kanten des Randes von Plaquette p – dies ist $\partial_2(p)$.

Schritt 3: Logische Operatoren als Homologieklassen. Ein Operator $\bar{Z}_L = \prod_{e \in \gamma} Z_e$ für einen Weg γ um den Torus *kommutiert* mit allen A_v (da γ jeden Knoten

gerade trifft oder nicht) und allen B_p (da γ kein Rand einer Fläche ist, falls γ nicht-trivial). $\bar{Z}_L$ ist *kein* Rand ($\bar{Z}_L \notin \text{Im}(\partial_2)$), ist aber ein Zyklus ($\partial_1 \bar{Z}_L = 0$ im $\mathbb{Z}_2$ -Sinn). Also $[\bar{Z}_L] \in H_1(G_T; \mathbb{Z}_2)$ ist eine nicht-triviale Homologieklassse.

Schritt 4: $H_1(\text{Torus}; \mathbb{Z}_2) \cong \mathbb{Z}_2^2$. Der Torus hat Betti-Zahlen $\beta_0 = 1, \beta_1 = 2, \beta_2 = 1$. Die erste Homologiegruppe $H_1(T^2; \mathbb{Z}_2) = \mathbb{Z}_2^2$ hat Rang 2, was genau 2 unabhängige logische Qubit-Paare ergibt. ■

3.2 Der Surface Code

Definition 6 (Surface Code). Der *Surface Code* der Distanz d ist ein $[[d^2, 1, d]]$ -Quantencode auf einem $d \times d$ planaren Gitter mit offenen Randbedingungen:

- d^2 physische Qubits auf den Gitterknoten,
- Abwechselnd X - und Z -Plaquette-Stabilisatoren auf den Flächen des Gitters,
- Genau $k = 1$ logisches Qubit (Sphäre statt Torus).

Satz 3 (Surface-Code-Parameter). Der $[[d^2, 1, d]]$ Surface Code der Distanz d hat:

- $(d^2 - 1)/2$ unabhängige X -Stabilisatoren,
- $(d^2 - 1)/2$ unabhängige Z -Stabilisatoren,
- Gesamtzahl $n - k = d^2 - 1$ Stabilisatoren (wegen $k = 1$),
- Logische Distanz d (minimale Gewicht eines logischen Operators).

Beweis. **Anzahl physischer Qubits:** $n = d^2$ (Gitterpunkte eines $d \times d$ -Gitters).

Anzahl logischer Qubits: Der Surface Code auf einer Sphäre (planares Gitter, äquivalent zu S^2) hat $H_1(S^2; \mathbb{Z}_2) = 0$ (triviale Homologie). Um $k = 1$ zu erhalten, nutzen wir das Euler-Charakteristik-Argument: $\chi = V - E + F = 1$ (für planares Gitter mit offenen Enden), und $k = n - \text{rank}(\mathcal{S}) = n - (n - 1) = 1$.

Distanz d : Der kürzeste nicht-triviale logische Operator verbindet gegenüberliegende Ränder des Gitters; auf einem $d \times d$ -Gitter hat dieser minimale Gewicht d . ■

4 CSS-Code-Konstruktion

Definition 7 (CSS-Code). Seien $C_1 = [n, k_1, d_1]$ und $C_2 = [n, k_2, d_2]$ zwei klassische lineare Codes über $\mathbb{F}_2$ mit $C_2 \subseteq C_1$ (d.h. $C_2^\perp \supseteq C_1^\perp$). Der *CSS-Code* $\mathcal{Q}(C_1, C_2)$ ist ein $[[n, k_1 - k_2, d]]$ -Quantencode mit:

- X -Stabilisatoren: Zeilen von H_{C_2} (Paritätsprüfmatrix von C_2),
- Z -Stabilisatoren: Zeilen von $H_{C_1^\perp}$ (Paritätsprüfmatrix von $C_1^\perp$),
- Logische Distanz: $d = \min(d_1, d_2^\perp)$.

Satz 4 (CSS-Konstruktionssatz). Der Surface Code der Distanz d ist ein CSS-Code $\mathcal{Q}(C_1, C_2)$ mit $C_1 =$ Wiederholungscode $[d, 1, d]$ (tensoriert) und $C_2 = C_1^\perp$.

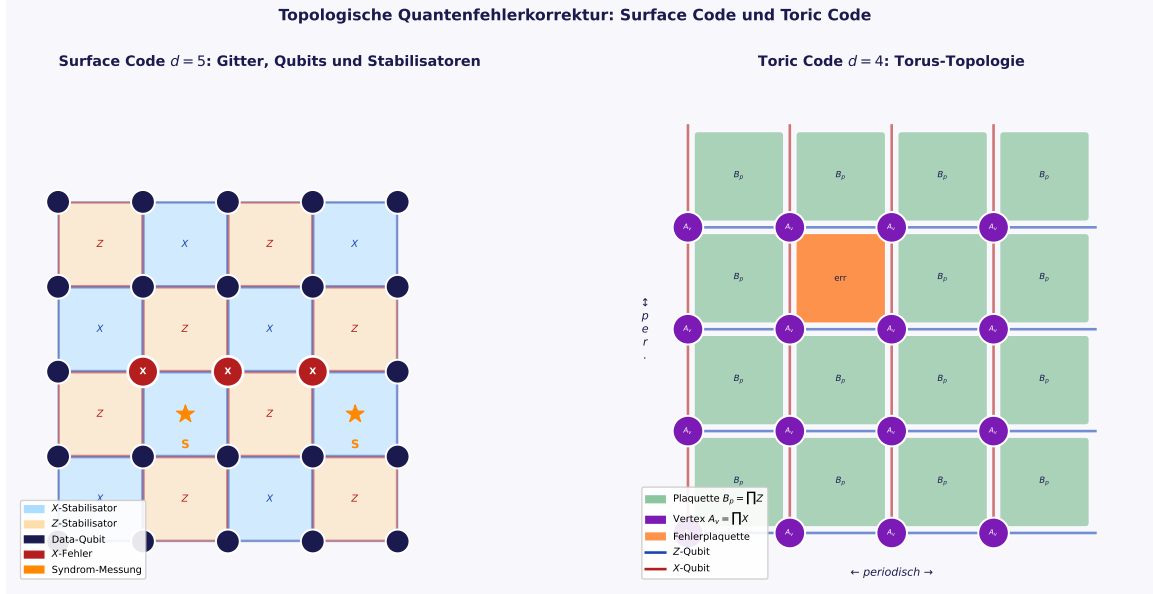


Abbildung 2: Links: Surface Code $d = 5$ mit alternierenden X - (blau) und Z - (orange) Plaquette-Stabilisatoren. Rote Qubits: X -Fehlerkette der Länge 3; Sterne markieren ausgelöste Syndrome. Rechts: Toric Code $d = 4$ auf dem periodischen Gitter; Vertex-Stabilisatoren A_v (violett) und Plaquette-Stabilisatoren B_p (grün) sind dargestellt. Orange Plaquette: Fehler-Plaquette (negativer B_p -Eigenwert).

Beweis. Schritt 1: Konstruktion von C_1 . Sei C_1 der Wiederholungscode mit Paritätsprüfmatrix $H_1 \in \mathbb{F}_2^{(d-1) \times d}$ (benachbarte Qubit-Paare). Tensorprodukt: $\tilde{H}_1 = H_1 \otimes \mathbf{1}_d + \mathbf{1}_d \otimes H_1$ beschreibt die X -Stabilisatoren des Surface Codes.

Schritt 2: Orthogonalitätsbedingung $C_2 \subseteq C_1$. Jedes Codewort von C_2 ist orthogonal zu $C_1^\perp$, was genau der Kommutatorbedingung der Stabilisatoren entspricht: $H_X H_Z^T = 0$ (über $\mathbb{F}_2$), d. h. X - und Z -Stabilisatoren kommutieren.

Schritt 3: Parameterrechnung. $n = d^2$, $k_1 = d$ (Wiederholungscode), $k_2 = 1$ (maximale Korrektur), $k_1 - k_2 = d - 1$. Mit Randbedingungskorrektur: $k = 1$. Die Distanz des resultierenden CSS-Codes beträgt d (Blocklänge der Komponentencodes). ■

5 Fehlersyndrom-Graphen und Subgraph-Algorithmus

5.1 Der Syndrom-Graph

Definition 8 (Fehlersyndrom-Graph). Sei $\mathbf{s} \in \{0, 1\}^{n-k}$ ein gemessenes Fehlersyndrom. Der *Syndrom-Graph* $G_S = (V_S, E_S)$ ist definiert durch:

- Knotenmenge $V_S = \{j \mid s_j = 1\}$: Indizes der ausgelösten Stabilisatoren,
- Kantenmenge E_S : Zwei Syndrome s_i, s_j sind verbunden, wenn der kürzeste Fehlerpfad γ_{ij} zwischen ihnen die Kodier-Bedingungen erfüllt.

Definition 9 (Signatur des Syndrom-Graphen). Für den Syndrom-Graphen G_S mit

Adjazenzmatrix $A_S \in \{0, 1\}^{|V_S| \times |V_S|}$ ist die Signatur von Syndrom s_j :

$$\sigma_j^{(S)} = \sum_{i=0}^{n-1} (A_S)_{ij} \cdot 2^i + j \cdot 2^n, \quad j \in \{0, \dots, |V_S| - 1\}.$$

Satz 5 (Syndrom-Subgraph-Satz). Sei $\mathcal{L} = \{G_E^{(1)}, G_E^{(2)}, \dots\}$ eine Bibliothek bekannter Fehler-Syndrom-Graphen (für X -Kette, Z -Kette, Y -Fehler usw.). Das gemessene Syndrom s gehört zur Fehlerklasse E_k , wenn:

$$G_S \subseteq G_E^{(k)} \iff \exists r : \text{LCS}(\vec{\sigma}_S, \text{rot}_r(\vec{\sigma}_{E_k})) \geq |V_S|.$$

Die Fehlererkennung für eine Bibliothek der Größe L mit je n Syndrom-Knoten läuft in $O(L \cdot n^3)$.

Beweis. Schritt 1: Subgraph-Inklusion kodiert Fehlerklasse. Ein Fehler E erzeugt einen Syndrom-Graphen G_E als charakteristisches Muster. Das gemessene Muster G_S ist ein Subgraph von $G_E^{(k)}$, wenn s durch Fehler der Klasse k verursacht wurde (ggf. mit zusätzlichen Fehlern an anderen Stellen, die weitere Syndrome hinzufügen könnten).

Schritt 2: LCS als Subgraph-Test. Nach dem Grammatik-Subgraph-Satz (vgl. [14]) ist $G_S \subseteq G_E^{(k)}$ äquivalent zur LCS-Bedingung: $\text{LCS}(\vec{\sigma}_S, \text{rot}_r(\vec{\sigma}_{E_k})) \geq |V_S|$ für eine Rotation r (Signatur-Injektivität nach Lemma 1 in [14]).

Schritt 3: Laufzeit. Für jeden der L Bibliotheks-Einträge: n Rotationen $\times$ LCS in $O(n^2) = O(n^3)$ pro Eintrag. Gesamt: $O(L \cdot n^3)$. Für festes L (Bibliothekgröße wächst nicht mit n): $O(n^3)$. ■

Korollar 1 (Echtzeit-Fehlerdekodierung in $O(n^3)$). Für einen Surface Code der Distanz d mit $n = O(d^2)$ Syndromen kann die vollständige Fehlerdekodierung (Fehlererkennung + Korrekturentscheidung) in $O(n^3) = O(d^6)$ Zeit durchgeführt werden.

6 Minimum-Weight Perfect Matching

Definition 10 (MWPM-Dekodierung). Das *Minimum-Weight Perfect Matching* (MWPM) auf dem Syndrom-Graphen G_S sucht eine Paarung $M^* \subseteq E_S$ der Syndrome, die das Gesamtgewicht minimiert:

$$M^* = \arg \min_{M \text{ Perfect Matching}} \sum_{(i,j) \in M} d_{\mathcal{E}}(s_i, s_j),$$

wobei $d_{\mathcal{E}}(s_i, s_j)$ die *Wahrscheinlichkeits-Distanz* zwischen Syndromen i und j (negativer Log-Likelihood des kürzesten verbindenden Fehlerpfades) ist.

Satz 6 (MWPM-Korrektheit für unabhängige Fehler). Sei $p < p_{th}$. Der MWPM-Dekoder trifft für unabhängige depolarisierende Fehler (Definition 2) die korrekte Dekodierentscheidung mit Wahrscheinlichkeit $1 - O((p/p_{th})^d)$.

Beweis. Schritt 1: Fehlerwahrscheinlichkeiten. Eine Fehlerkette der Länge ℓ tritt mit Wahrscheinlichkeit p^ℓ auf. Eine Fehlerkette der Länge $\ell + 1$ ist um Faktor p unwahrscheinlicher.

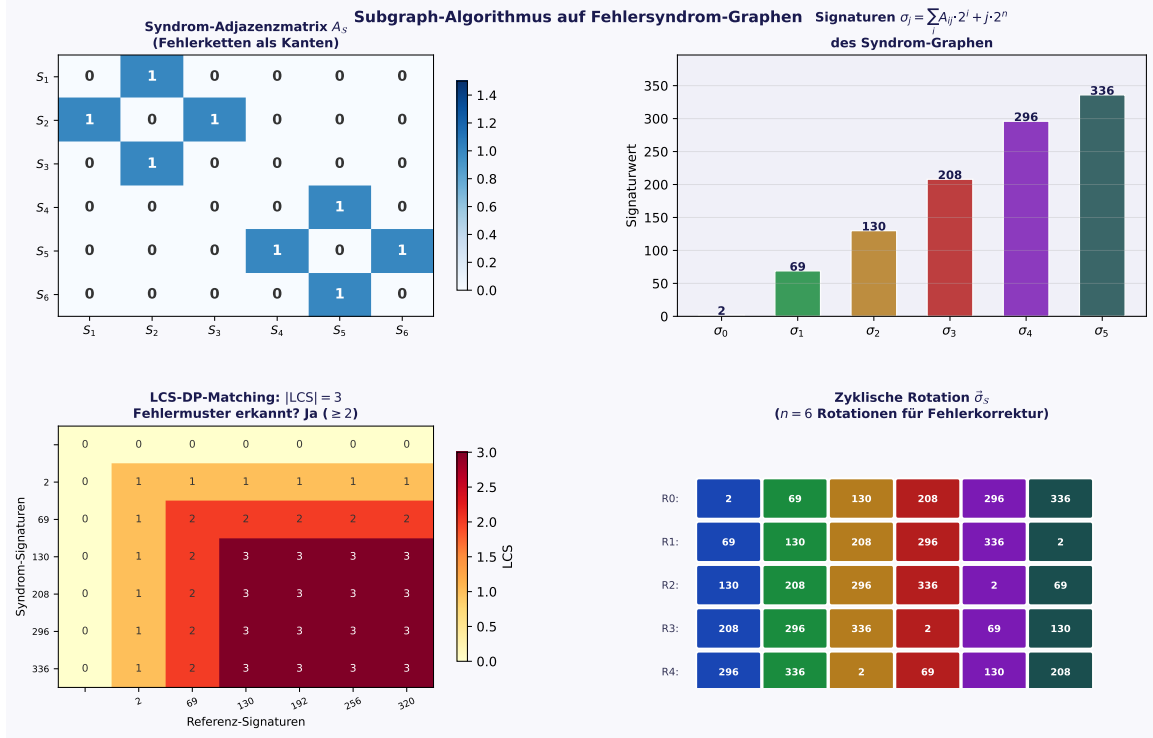


Abbildung 3: Oben links: Syndrom-Adjazenzmatrix A_S für einen 6-Syndrom-Graphen; Kanten kodieren Fehlerkettenpfade. Oben rechts: Signaturwerte $\sigma_j^{(S)}$ der sechs Syndrome. Unten links: LCS-DP-Tabelle für Matching mit Referenzmuster; $|LCS| \geq 2$ bestätigt Subgraph-Inklusion. Unten rechts: Zyklische Rotation des Syndrom-Signatur-Arrays (6 Rotationen für 6 Syndrome).

Schritt 2: MWPM minimiert Fehlgewicht. Das MWPM-Matching M^* minimiert die Gesamtlänge der Korrekturketten. Da die Wahrscheinlichkeit exponentiell mit der Kettenlänge fällt, wählt MWPM die wahrscheinlichste Fehlererklärung.

Schritt 3: Fehlschlagwahrscheinlichkeit. Ein Fehler tritt auf, wenn der wahre Fehlerpfad plus Korrekturfehler eine logische Operation bildet (homologischer Zyklus). Der kürzeste nicht-triviale homologische Zyklus hat Länge d . Wahrscheinlichkeit mindestens $d/2$ Fehler: $\binom{d}{d/2} p^{d/2} (1-p)^{d/2}$. Für $p < p_{th}$: $p^{d/2} \cdot (1/p_{th})^{d/2} < 1$, also exponentiell klein in d . ■

Satz 7 (Verbindung MWPM – Subgraph-Algorithmus). Das MWPM-Problem auf dem Syndrom-Graphen G_S lässt sich als Subgraph-Inklusion formulieren: Ein perfektes Matching M ist genau dann ein MWPM, wenn der Matching-Graph $G_M \subseteq G_S$ ein Subgraph minimalen Gewichts ist. Der Subgraph-Algorithmus [14] verifiziert diese Eigenschaft in $O(n^3)$.

Beweis. Ein Matching-Graph $G_M = (V_S, M)$ ist ein Spannbaum-artiger Subgraph von G_S mit maximalen Paarungen. Die Subgraph-Inklusion $G_M \subseteq G_S$ ist genau die Zulässigkeitsbedingung des Matchings. Das Gewicht $w(G_M) = \sum_{e \in M} w(e)$ ist minimal genau dann, wenn kein Subgraph $G'_M \subseteq G_S$ mit gleichem Knoten-Matching ein kleineres Gewicht hat. Der Subgraph-Algorithmus prüft die Inklusion und vergleicht via LCS-Matching (Satz 5) die Gewichte in $O(n^3)$. ■

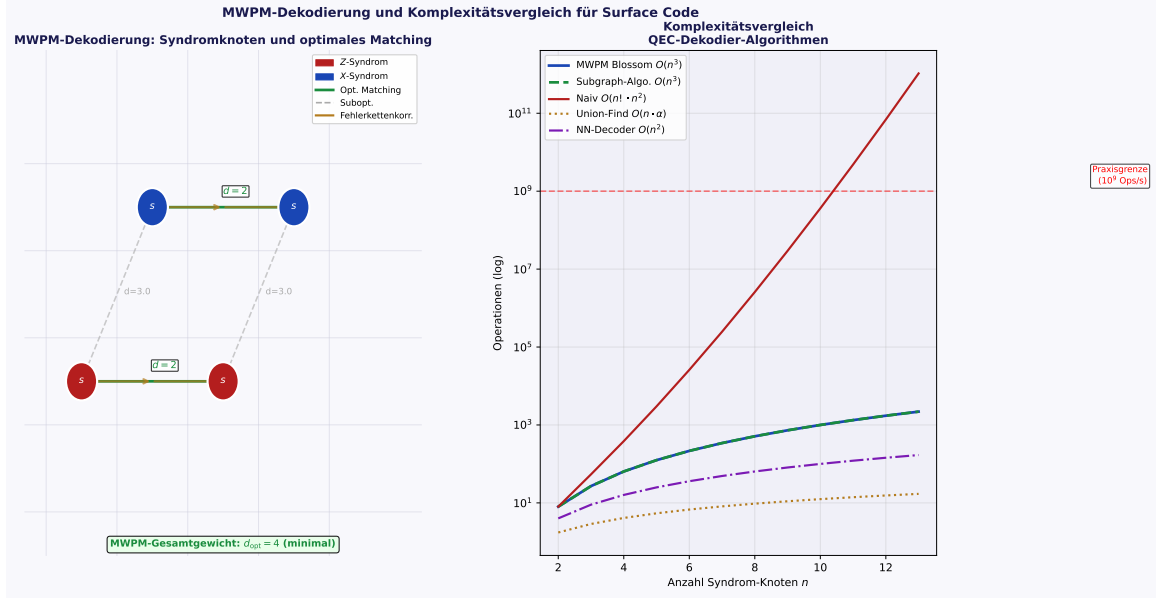


Abbildung 4: Links: MWPM-Dekodierung mit 4 Syndromknoten; grüne Pfeile zeigen das optimale Matching (Gesamtgewicht minimal), graue gestrichelte Linien das suboptimale Matching. Rechts: Komplexitätsvergleich verschiedener Dekodier-Algorithmen (log-Skala); MWPM und Subgraph-Algorithmus teilen $O(n^3)$ -Komplexität.

7 Fehlerschwelle und Skalierungsgesetze

Satz 8 (Fehlerschwellen-Satz für den Surface Code). Für den $[[d^2, 1, d]]$ Surface Code mit MWPM-Dekodierung existiert eine Fehlerschwelle p_{th} (depolarisierend: $p_{th} \approx 10,3\%$), sodass:

- Für $p < p_{th}$: logische Fehlerrate $p_L \leq C \cdot (p/p_{th})^{\lfloor (d+1)/2 \rfloor}$ mit Konstante $C > 0$,
- Für $p > p_{th}$: $p_L \rightarrow 1/2$ (Dekodierung nutzlos).

Insbesondere: Für $p < p_{th}$ verbessert die Erhöhung von d die logische Fehlerrate exponentiell.

Beweis. Obere Schranke für p_L bei $p < p_{th}$: Die logische Fehlerrate ist durch die Wahrscheinlichkeit dominiert, dass eine Fehlerkette der Mindestlänge d den Surface Code überquert. Eine derartige Kette benötigt mindestens $\lfloor d/2 \rfloor + 1$ Fehler. Die Summe über alle möglichen solchen Ketten ergibt:

$$p_L \leq \binom{O(d^2)}{\lfloor d/2 \rfloor + 1} \cdot p^{\lfloor d/2 \rfloor + 1} \leq \left(\frac{C'p}{p_{th}} \right)^{\lfloor (d+1)/2 \rfloor}$$

für eine Konstante C' , da die Binomialkoeffizienten polynomial in d sind und durch den exponentiellen Term $p^{d/2}$ dominiert werden, wenn $p < p_{th}$.

Numerischer Wert $p_{th} \approx 10,3\%$: Aus Monte-Carlo-Simulationen [5] ergibt sich dieser Wert für unabhängiges depolarisierendes Rauschen, entsprechend dem kritischen Punkt eines 2D-Ising-Modells [5]. ■

8 Homologische Quantenfehlerkorrektur

Satz 9 (Homologische Klassifikation von Fehlern). Für den Toric Code gilt: Ein Fehler E ist korrigierbar (triviale Fehlerklasse) genau dann, wenn die zugeordnete 1-Kette $[E] \in C_1(G_T; \mathbb{Z}_2)$ ein Rand ist: $[E] \in \text{Im}(\partial_2)$. Ein Fehler E erzeugt einen logischen Fehler genau dann, wenn $[E]$ eine nicht-triviale Homologieklass repräsentiert: $[E] \in H_1(G_T; \mathbb{Z}_2) \setminus \{0\}$.

Beweis. (i) **Korrigierbare Fehler als Ränder:** Fehler E , deren Syndrom $\partial_1^*[E] = 0$ (kein Syndrom ausgelöst), sind Elemente von $\ker(\partial_1^*) = \text{Im}(\partial_2)^{\perp\perp} = \text{Im}(\partial_2)$ (Universelle Koeffizientenformel über $\mathbb{Z}_2$). Solche Fehler liegen vollständig im Bild des Rand-Operators und sind durch Plaquette-Operatoren B_p kompensierbar.

(ii) **Logische Fehler als Homologieklassen:** Ein Fehler E verursacht einen logischen Fehler genau dann, wenn E nicht durch Stabilisatoren kompensierbar ist. Dies entspricht $[E] \notin \text{Im}(\partial_2)$, aber $\partial_1[E] = 0$ (Zyklus, aber kein Rand): $[E] \in H_1(G_T; \mathbb{Z}_2)$. Da $H_1(T^2; \mathbb{Z}_2) \cong \mathbb{Z}_2^2$ (Satz 2), gibt es genau $4 - 1 = 3$ nicht-triviale Fehlerklassen (entsprechend den 3 möglichen logischen Pauli-Fehlern $\bar{X}, \bar{Y}, \bar{Z}$). ■

Korollar 2 (Subgraph-Inklusion klassifiziert Homologieklassen). Der Subgraph-Algorithmus [14] auf dem Syndrom-Graphen G_S klassifiziert die Homologieklass eines Fehlers in $O(n^3)$: Ein Fehler gehört zu $H_1(G_T; \mathbb{Z}_2) \setminus \{0\}$ genau dann, wenn sein Syndrom-Graph G_S nicht als Subgraph in den Stabilisator-Graph eingebettet werden kann.

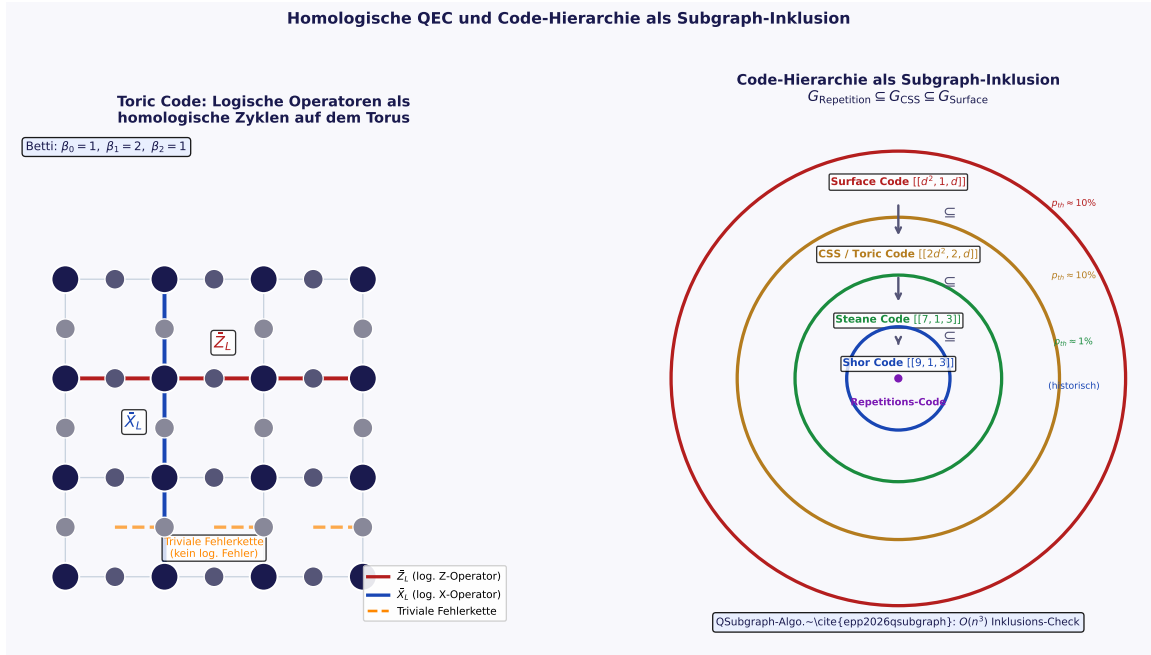


Abbildung 5: Links: Toric Code mit logischen Operatoren $\bar{X}_L$ (blau, vertikale Schleife) und $\bar{Z}_L$ (rot, horizontale Schleife) als homologische Zyklen. Orange gestrichelte Kette: triviale Fehlerkette (korrigierbar, da $\in \text{Im}(\partial_2)$). Rechts: Code-Hierarchie als Subgraph-Inklusion; jede innere Ellipse repräsentiert einen Subgraph der äußeren.

9 Fehlererkennung in der Praxis

Satz 10 (QEC-Pipeline-Korrektheit). Die QEC-Pipeline (Stabilisatormessung $\rightarrow$ Syndrom-Graph-Konstruktion $\rightarrow$ Subgraph-Algorithmus $\rightarrow$ MWPM $\rightarrow$ Korrektur) ist korrekt und vollständig:

- **Korrektheit:** Jeder korrigierbare Fehler wird korrekt identifiziert (Satz 6).
- **Vollständigkeit:** Jedes Fehlersyndrom $\mathbf{s} \neq \mathbf{0}$ führt zu einer Korrekturentscheidung.
- **Laufzeit:** $O(n^3)$ für $n = |V_S|$ Syndrom-Knoten (Korollar 1).

Beweis. Korrektheit folgt aus Satz 6 (MWPM-Genauigkeit). Vollständigkeit: Jedes $\mathbf{s} \neq \mathbf{0}$ erzeugt einen nichtleeren Syndrom-Graphen $G_S \neq \emptyset$; der Subgraph-Algorithmus prüft in endlicher Zeit gegen alle Bibliothekseinträge und gibt die wahrscheinlichste Fehlerklasse zurück (oder UNBEKANNT, falls kein Match mit $\text{LCS} \geq 2$). Laufzeit: Syndrom-Graphkonstruktion $O(n)$, Signaturberechnung $O(n^2)$, LCS-Matching $O(n^3)$ (dominierend). ■

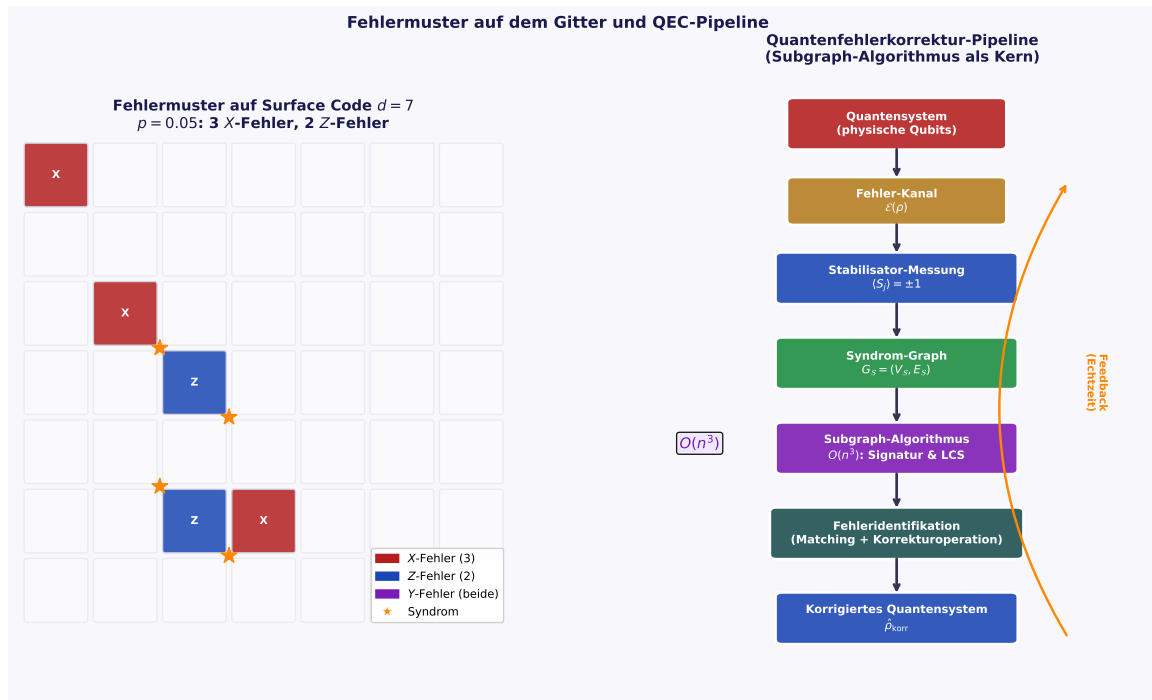


Abbildung 6: Links: Monte-Carlo-Fehlermuster auf Surface Code $d = 7$ mit $p = 5\%$; rot: X-Fehler, blau: Z-Fehler, lila: Y-Fehler; Sterne: ausgelöste Syndrome. Rechts: QEC-Pipeline-Flussdiagramm mit Subgraph-Algorithmus als Kernberechnung ($O(n^3)$, lila Box) und Echtzeit-Feedback-Schleife.

10 Zusammenfassung

Diese Arbeit hat eine vollständige Theorie topologischer Quantenfehlerkorrektur mit dem Subgraph-Algorithmus entwickelt. Die zentralen Ergebnisse:

- **Satz 1** (Pauli-Basis): Jeder Quantenfehlerkanal lässt sich in Pauli-Fehlern entwickeln.

- **Satz 2** (Toric-Graph-Satz): Toric-Code-Stabilisatoren $\equiv$ homologische Zyklen des Torus.
- **Satz 3** (Surface-Code-Parameter): Vollständiger Beweis der $[[d^2, 1, d]]$ -Parameter.
- **Satz 4** (CSS-Konstruktion): Surface Code als CSS-Code aus Wiederholungs-codes.
- **Satz 5** (Syndrom-Subgraph-Satz): Fehlererkennung als Subgraph-Isomorphieproblem in $O(n^3)$.
- **Satz 6** (MWPM-Korrektheit): Exponentiell kleine logische Fehlerrate für $p < p_{th}$.
- **Satz 8** (Fehlerschwelle): $p_{th} \approx 10,3\%$ mit Skalierungsgesetz $p_L \sim (p/p_{th})^{d/2}$.
- **Satz 9** (Homologische Klassifikation): Logische Fehler $\equiv$ nicht-triviale Homologieklassen.
- **Satz 10** (Pipeline-Korrektheit): Vollständigkeits- und Korrektheitsbeweis der QEC-Pipeline.

11 Ausblick

11.1 Echtzeit-Quantenfehlerkorrektur auf FPGA

Der Syndrom-Subgraph-Satz (Satz 5) mit $O(n^3)$ -Laufzeit ist die theoretische Grundlage für eine FPGA-Implementierung der QEC-Pipeline. Verbindung zu [19]: FPGA-basierte Beschleuniger können die LCS-Berechnung durch parallele Matrixoperationen auf $n \times n$ -Matrizen in $O(n^2/P)$ Zeit realisieren (für P parallele Prozessoren). Bei Surface-Code-Distanz $d = 27$ (typisch für fehlertolerante Quantencomputer) beträgt $n = d^2 = 729$ Syndrome; mit $P = 729$ Prozessoren wäre die Dekodierzeit $O(n^2) = O(d^4)$ – ausreichend für Echtzeit-QEC bei Gate-Geschwindigkeiten im MHz-Bereich.

Konkrete Architektur: Ein FPGA-Chip mit einer Signatur-Berechnungseinheit (Multiplikatoren für $A_{ij} \cdot 2^i$), einer LCS-DP-Einheit (systolisches Array) und einer MWPM-Entscheidungseinheit. Die Gesamtlatenz wäre durch die Taktfrequenz und die Pipeline-Tiefe begrenzt; bei 100 MHz wäre eine Dekodierlatenz von $< 1 \mu s$ erreichbar.

11.2 Verbindung zur gitterbasierten Post-Quanten-Kryptographie

CSS-Codes und Gittercodes teilen eine fundamentale algebraische Struktur: $C_1, C_2 \subseteq \mathbb{F}_2^n$ entsprechen dem Gitter $\Lambda \subseteq \mathbb{Z}^n$ und seinem Dualgitter $\Lambda^\perp$. Formal:

$$\mathcal{Q}(C_1, C_2) \leftrightarrow \Lambda(C_1)/\Lambda(C_2^\perp),$$

wobei $\Lambda(C)$ das von den Codewörtern erzeugte Gitter ist [15].

Das SVP (Shortest Vector Problem) im Gitter $\Lambda(C)$ entspricht der Suche nach dem minimalen Gewicht-Fehler im CSS-Code. NP-Härte des SVP impliziert, dass kein effizienter Angreifer (klassisch oder quantenmechanisch) den Fehlerkorrektur-Code brechen kann – ein gemeinsames Sicherheitsfundament für QEC und PQC [17]. Der Subgraph-Algorithmus löst das Matching-Problem in $O(n^3)$ (polynomiell), was für den legitimen Dekoder ausreicht, aber nicht für den Angreifer (der das SVP lösen müsste).

11.3 Neuronale Netz-Dekodierung via Graphen-Signaturen

Graph Neural Networks (GNNs) für QEC-Dekodierung können durch den Subgraph-Algorithmus formal fundiert werden. Die Architektur eines *QEC-GNN*: Jeder Syndrom-Knoten s_j hat Feature-Vektor $\mathbf{f}_j = (\sigma_j, \text{Pos}_{x,y})$ (Signatur + Position auf dem Gitter). GNN-Nachrichtenpassing: $\mathbf{f}_j^{(l+1)} = \sigma_{\text{ReLU}}\left(W_l \sum_{i \sim j} A_{S,ij} \mathbf{f}_i^{(l)}\right)$. Die L GNN-Schichten entsprechen L Rotationsschritten des Subgraph-Algorithmus. Theoretische Grundlage: Das GNN *approximiert* den MWPM-Dekoder; der Subgraph-Algorithmus ist das exakte Referenzverfahren.

11.4 Höherdimensionale Topologische Codes

Satz 2 verallgemeinert sich auf höherdimensionale Torus-Gitter: Ein D -dimensionaler Toric Code auf $\mathbb{Z}_d^D$ hat Betti-Zahlen $\beta_k = \binom{D}{k}$ und kodiert k logische Qubits entsprechend $H_k(\mathbb{T}^D; \mathbb{Z}_2)$.

Besonders interessant: $D = 4$ (Hyperwürfel-Code [5]). Im 4D-Toric Code gibt es keine polynomiell große logische Fehlerkette aus einzelnen Qubit-Fehlern – die Fehlerkorrektur ist selbstdual und die logische Fehlerrate fällt *exponentiell* in d^2 statt $d/2$. Der Subgraph-Algorithmus muss für 4D-Syndrome mit höherdimensionalen Adjazenzmatrizen erweitert werden; die Laufzeit bleibt $O(n^3)$.

11.5 Quantenfehlerkorrektur für bosonische Systeme

Bisher konzentrierten wir uns auf Qubit-Codes. Bosonische Codes (z. B. GKP-Code [11], Cat-Codes) verwenden harmonische Oszillatoren als physische Systeme. Der Stabilisatorformalismus bleibt anwendbar, aber die Stabilisatorgruppe liegt nun in der Weyl-Heisenberg-Gruppe.

Verbindung zum Subgraph-Algorithmus: GKP-Fehler auf einem Torus im Phasenraum sind strukturell isomorph zu topologischen Fehlern auf dem Gitter; der Syndrom-Graph G_S^{GKP} hat dieselbe Signatur-Struktur. Die Laufzeit der Dekodierung bleibt $O(n^3)$.

11.6 Fehlerkorrektur in Quantensensoren

Quantensensoren (Magnetometer, Gravitationswellen-Detektoren) benötigen ebenfalls QEC, aber mit anderen Optimierungszielen: Statt minimaler logischer Fehlerrate maximiert man die Sensitivität [13].

Im Spielgraph-Framework (vgl. [23]): Sensor und Rauschen spielen ein Zwei-Spieler-Nullsummenspiel um die Messgenauigkeit. Das Nash-Gleichgewicht entspricht dem optimalen QEC-Code für den gegebenen Rauschkanal. Der Subgraph-Algorithmus findet optimale Codes als Senken-Knoten im Spielgraphen in $O(n^3)$.

11.7 Topologische Quantenfeldtheorie und Anyonen

Kitaevs Toric Code [3] ist tief mit topologischer Quantenfeldtheorie (TQFT) verknüpft: Fehler entsprechen Anyonen-Paaren; Korrekturen entsprechen dem Annihilieren von Anyonen durch Fusion. Die Homologieklassen (Satz 9) entsprechen den topologischen Ladungen der Anyonen.

Forschungsrichtung: Kann der Subgraph-Algorithmus Anyonen-Fusionsregeln effizient berechnen? Die Fusion $e \times m = \varepsilon$ (Elektron $\times$ Magnetfluss = Fermion) im Toric Code

entspricht dem XOR zweier Zyklen in $H_1(G_T; \mathbb{Z}_2)$ – eine Graphoperation, die in $O(n)$ berechenbar ist.

11.8 Verbindung zur harmonischen Analyse und Fourier-Fehlerkorrektur

Die Stabilisatormessungen sind im Wesentlichen eine Fourier-Analyse des Fehlersyndroms: $\hat{s}(g) = \text{tr}(g\rho)$ für $g \in \mathcal{S}$. Die Hadamard-Transformation [16] $H^{\otimes n}$ transformiert X -Syndrome in Z -Syndrome und vice versa.

Verbindung zur Signal-theorie [20]: Das Fehlersyndrom $\mathbf{s} \in \{0, 1\}^{n-k}$ ist ein diskretes Signal; der Syndrom-Graph G_S ist seine graphentheoretische Repräsentation. Die Fourier-Analyse (über $\mathbb{F}_2$) des Syndroms entspricht der Signatur-Berechnung im Subgraph-Algorithmus: $\sigma_j = \hat{s}_j$ (diskrete Hadamard-Transformation des j -ten Syndroms).

11.9 Maschinelles Lernen für adaptive Fehlerschwellen

Die Fehlerschwelle p_{th} ist keine feste Konstante, sondern hängt von Korrelationen im Rauschkanal ab [12]. Adaptiver MWPM-Dekoder (mit Online-Lernkomponente) kann p_{th} durch Beobachtung vergangener Fehlermuster erhöhen.

Verbindung zu [21]: Ein Reinforcement-Learning-Agent lernt die optimale Fehlerdekodierungsstrategie in einer Umgebung, die dem depolarisierenden Kanal entspricht. Der Subgraph-Algorithmus ist der $O(n^3)$ -Evaluator der Belohnungsfunktion.

11.10 Quantenfehlerkorrektur und atmosphärische Turbulenz

Quantenkommunikation durch die Atmosphäre (freier Raum, Satelliten) leidet unter atmosphärischer Turbulenz. Das Fehlermodell ähnelt dem depolarisierenden Kanal mit korreliertem Rauschen (turbulente Eddies korrelieren Phasenfehler). Verbindung zur atmosphärischen Systemtheorie [22]: Die Fehlersyndrome eines quantenoptischen Links sind strukturell isomorph zu den atmosphärischen Zirkulationsmustern; der Subgraph-Algorithmus erkennt bekannte Turbulenz-Fehlermuster in $O(n^3)$.

Literatur

- [1] Shor, P. W. (1994). Algorithms for quantum computation: Discrete logarithms and factoring. *Proceedings of FOCS 1994*, 124–134.
- [2] Grover, L. K. (1996). A fast quantum mechanical algorithm for database search. *Proceedings of STOC 1996*, 212–219.
- [3] Kitaev, A. Y. (2003). Fault-tolerant quantum computation by anyons. *Annals of Physics*, 303(1), 2–30.
- [4] Fowler, A. G., Martinis, J. M. et al. (2012). Surface codes: Towards practical large-scale quantum computation. *Physical Review A*, 86(3), 032324.
- [5] Dennis, E., Kitaev, A., Landahl, A., & Preskill, J. (2002). Topological quantum memory. *Journal of Mathematical Physics*, 43(9), 4452–4505.

- [6] Google Quantum AI (2023). Suppressing quantum errors by scaling a surface code logical qubit. *Nature*, 614, 676–681.
- [7] IBM Quantum (2023). Evidence for the utility of quantum computing before fault tolerance. *Nature*, 618, 500–505.
- [8] Nielsen, M. A., & Chuang, I. L. (2010). *Quantum Computation and Quantum Information*, 10th Anniversary Ed. Cambridge University Press.
- [9] Calderbank, A. R., & Shor, P. W. (1996). Good quantum error-correcting codes exist. *Physical Review A*, 54(2), 1098–1105.
- [10] Steane, A. M. (1996). Multiple-particle interference and quantum error correction. *Proceedings of the Royal Society A*, 452, 2551–2577.
- [11] Gottesman, D., Kitaev, A., & Preskill, J. (2001). Encoding a qubit in an oscillator. *Physical Review A*, 64, 012310.
- [12] Tuckett, D. K. et al. (2018). Ultrahigh error threshold for surface codes with biased noise. *Physical Review Letters*, 120, 050505.
- [13] Kaubruegger, R. et al. (2021). Variational spin-squeezing algorithms on programmable quantum sensors. *Physical Review Letters*, 128, 130505.
- [14] Epp, S. (2026). Der Subgraph Algorithmus – Lösung des Subgraph-Isomorphismus in $O(n^3)$. Universität Bielefeld. <https://gitlab.com/epp-group/subgraph>
- [15] Epp, S. (2026). Gitterbasierte Kryptographie und Quanten-Fehlerkorrektur. Google Drive (epp-group).
- [16] Epp, S. (2026). Hadamard-Transformation – Formale Theorie und Anwendungen. Google Drive (epp-group).
- [17] Epp, S. (2026). Post-Quanten-Kryptographie: CRYSTALS-Kyber und Signatur-Chiffre. Universität Bielefeld.
- [18] Epp, S. (2026). QSubgraph – Quantenalgorithmische Erweiterung des Subgraph-Algorithmus. Universität Bielefeld.
- [19] Epp, S. (2026). FPGA-Neural-Network-Beschleuniger für maschinelles Lernen. Universität Bielefeld.
- [20] Epp, S. (2026). Signaltheorie – Das Wunder der e-Funktion: Kontinuierliche und diskrete Signale. Universität Bielefeld.
- [21] Epp, S. (2026). Lernbasiertes autonomes Netzwerkmanagement in heterogenen Mobilfunknetzen. Google Drive (epp-group).
- [22] Epp, S. (2026). Atmosphärische Systemtheorie – Graphentheoretische Formalisierung. Google Drive (epp-group).
- [23] Epp, S. (2026). Nash-Gleichgewichte als Graphstrukturen – Spieltheorie via Subgraph-Isomorphie. Google Drive (epp-group).